University of Prince Edward Island

An Introduction to Instanced Rendering in OpenGL

Ryan Campbell
0335084
Mr. Gordon Wood
April 16, 2024

Introduction

Efficient rendering techniques are essential to be able to render graphic intensive programs. This paper provides an introduction to one of these techniques known as instanced rendering in OpenGL. The paper lays out the problem that instanced rendering solves, along with multiple approaches on how to implement this technique with varying degrees of complexity. Later on there is an explanation of the various instanced drawing functions supplied by OpenGL. The research is concluded with a test case measuring the performance of the approaches that were discussed.

The Problem

Graphics processing unit (GPU) instancing is a rendering technique to efficiently render identical geometry many times throughout a scene [1]. A common example of this could be grass or bushes which are generally made up of relatively few vertices but where hundreds or thousands of instances of them may need to be rendered, where the only difference is basic transformations such as scale, rotation and position [1]. Each of these instances can be rendered extremely fast but the overhead of having thousands of draw calls, each of which sends information across the central processing unit (CPU) - GPU bus is extremely slow in comparison [1]. Instancing is the solution to this problem which allows the rendering of multiple objects with the same mesh and material, but different transforms in one draw call [1].

Approach 1: The Naive Approach

The naive approach to rending many instances of the same object will be explained first to help illustrate the issue at hand. The goal for the following examples is to render many instances of a cube. In the simplest form, to render a cube, 36 indices, and eight vertices, with each vertex only storing the position attribute information can be used. For each cube the world matrix can be calculated and passed to the vertex shader as a uniform to render the individual cube. Doing this inside a for loop many cubes can be rendered. The code and shader to achieve this may look something like this:

```
//shader code
in vec4 a_position;
uniform mat4 worldMatrix;
uniform mat4 mVP;

void main(){
    gl_Position =  mVP * worldMatrix * a_position;
}
```

Figure 1.1: Vertex shader code for the naive rendering approach [1].

```
// upload uniforms and draw cubes
for (const auto& cube : cubeList) {
    cube.configurePipeline(); // bind VAO, textures, uniforms
    glDrawElements(GL_TRIANGLES, numIndices, GL_UNSIGNED_SHORT, 0);
}
```

Figure 1.2: Rendering code for the naive approach. [1]

At a first glance this approach seems viable but it is very inefficient once you start rendering a few hundred or few thousand cubes, depending on your hardware. Each time a cube is rendered the code to set up the material and vertex array object (VAO) is run [1]. These state changes along with many draw calls greatly decrease the performance in the scene being rendered. Since all the cubes being rendered are the same in terms of material and graphics pipeline configuration, these state calls are completely redundant, but still further slow down your program. The only thing that varies for each cube is its transform. OpenGL provides a way to render the same geometry with various transformations without these redundant state changes, and excessive draw calls. This leads to the next approach presented.

Approach 2: Instanced Draw Functions

For this example the two basic instanced draw call functions OpenGL provides will be explained. The remainder will be discussed later in the paper. *glDrawElementsInstanced* and *glDrawArraysInstanced* are the two basic instanced draw call functions that will be used instead of *glDrawElements* and *glDrawArrays* respectively [1]. These new functions take an extra parameter which specifies how many instances of the geometry that is to be rendered, allowing many instances to be rendered in one draw call [1]. OpenGL also provides a shader variable called *gl_InstanceID* [1]. *gl_InstanceID* is a built-in integer shader variable that is automatically incremented for each instance rendered per draw call [1]. That is, *gl_InstanceID* is a variable that starts at zero and increments once for each set of geometry being rendered in a draw call [1]. In this example to render many cubes this variable can be used to index into an array of matrices that represents the transform of each cube. Note: *gl_InstanceID* is also available in shaders when you do not use instanced draw calls but it will always be zero [1]. This means you can use the same shaders for instanced and non instanced draw calls if desired. The shader code will now have to be updated to take in a uniform array of matrices, which is indexed using *gl_InstanceID*. The render code will have to be updated to supply the shader with an array of world matrices and to use instanced draw calls. The code for this example may look something like this.

```
//shader code
in vec4 a_position;
uniform mat4 worldMatrices[255];
uniform mat4 mVP;
void main(){
    gl_Position =  mVP * worldMatrices[gl_InstanceID] * a_position;
}
```

Figure 2.1: Vertex shader code for the basic instanced rendering approach [1].

```
// Creating and uploading a list of world matrices
glm::vec3 pos(0.0f, 0.0f, 0.0f);
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < rows; ++j) {
        auto worldMat = glm::translate(glm::mat4(1.0f), pos);
        m_worldMatList.push_back(worldMat);
        pos.x += 2.0f;
    }
    pos.x = 0.0f;
    pos.z += 2.0f;
}
glUniformMatrix4fv(attribLoc, 255, GL_FALSE, glm::value_ptr(worldMatList[0]));

// rendering
configurePipeline(); // bind VAO, texture, uniforms.
glDrawElementsInstanced(GL_TRIANGLES, numIndices, GL_UNSIGNED_SHORT, 0, 255);
```

Figure 2.2: Rendering code for the basic instanced rendering approach [1].

This is the most basic approach that can be taken to use instanced rendering. Notice in the shader code that *gl_InstanceID* is being used to index into an array that holds the world matrices for the cubes. As can be seen in the rendering section, there is now only one draw call being sent to the GPU. This means all the data is sent over the slow CPU-GPU bus at once, and the pipeline is only configured one time, massively reducing redundant overhead. One problem with this method is that OpenGL only allows a certain amount of uniforms, which can vary depending on your hardware. 4x4 world matrices will quickly eat up this limited storage. For example, on my machine I was only able to store enough world matrices to render 255 cubes. This is an obvious issue if trying to efficiently render a scene with thousands of blades of grass, or bushes. Lucky OpenGL provides us with a way around this issue. This leads to the next topic of instanced arrays.

Approach 3: Instanced Arrays

Instanced arrays will now be used in order to solve the problem of limited uniform space described in the previous approach. Instanced arrays store vertex data but are updated with every instance being rendered as opposed to being updated for every vertex [1]. This data is stored in vertex buffer objects (VBO), just like normal vertex attributes [1]. Instead of uploading the world matrices as a uniform array, they can be stored in a vertex buffer. The only difference between this vertex buffer and ones that have been previously used is a call to *glVertexAttribDivisor* which is used to tell OpenGL to update the attribute per instance instead of per vertex [1]. Note: the second parameter in *glVertexAttribDivisor*, *divisor* specifies how often the vertex attribute should update [2]. If set to zero it updates per vertex, like normally expected [2]. If set to N, an integer greater than zero, it will be updated every N instances being rendered in that draw call [2]. For the example of rendering many cubes there is no longer a need to use *gl_InstanceID* to

access an array of world matrices in the vertex shader. The shader code takes in the world matrix just as it would any other vertex attribute. The code to create this buffer, and render cubes may look like this.

```
//shader code
in vec4 a_position;
in mat4 worldMatrices;
uniform mat4 mVP;

void main(){
    gl_Position =  mVP * worldMatrices * a_position;
}
```

Figure 3.1: Vertex shader code for the instanced array rendering approach [1]

```
// matrix vbo
// create m_worldMatList as before but now place contents in vertex buffer object
glGenBuffers(1, &m_matriceVbo);
glBindBuffer(GL_ARRAY_BUFFER, m_matriceVbo);
glBufferData(GL_ARRAY_BUFFER, sizeof(m_worldMatList), m_worldMatList, GL_STATIC_DRAW);

int matriceAttrib = glGetAttribLocation(m_program, "worldMatrices");
// bind matrix vbo
glBindBuffer(GL_ARRAY_BUFFER, m_matriceVbo);
glVertexAttribPointer(
    matriceAttrib, // Attribute location
    4, // Number of components
    GL_FLOAT, // Type of each component
    GL_FALSE, // Normalize?
    sizeof(GLfloat) * 4 * 4, // Stride
    0); // Offset
glEnableVertexAttribArray(matriceAttrib);
glVertexAttribDivisor(matriceAttrib, 1);
```

Figure 3.2: Setting up the instanced array to hold world matrices [1, 12].

```
configurePipeline();
glDrawElementsInstanced(GL_TRIANGLES, numIndices, GL_UNSIGNED_SHORT, 0, numCubes);
```

Figure 3.3: Rendering Code for the instanced arrays rendering approach [1].

Note: The maximum number of components a vertex attribute can have is four [5]. In the example above there is a 16 component matrix being uploaded as a vertex attribute. For this to work there needs to be four calls to both *glVertexAttribDivisor* and *glVertexAttribPointer*, one for each row of the 4x4 matrix (See appendix A for more details) [12]. As seen in the last line of code in figure 3.2 there is a call to *glVertexAttribDivisor*. The first parameter, *index,* specifies which attribute in the currently bound shader we are working with [2]. The second parameter, *divisor*, specifies the aforementioned attribute divisor. Here the attribute divisor is set to one so the world matrix will be updated per instance being rendered. This allows each vertex in a cube to use the same world matrix, specified as a vertex attribute in the shader. The maximum size this

buffer can be is much larger than the maximum size when using a uniform array. Rendering many thousands of instances of geometry in one draw call, without the worry of running out of uniform buffer memory is now possible.

Overview Of Non-Indexed Instanced Draw Functions

Non-indexed draw function	Notes
glDrawArraysInstancedBaseInstance(GLenum mode, GLint first, GLsizei count, GLsizei instancecount, GLuint baseinstance)	● Draws instanceCount sets of specified geometry. ● Starts retrieving instanced vertex attributes from *baseinstance* location in VBO. ● baseInstance does not affect the value of *gl_InstanceID*
glDrawArraysInstanced(GLenum mode, GLint first, GLsizei count, GLsizei instancecount)	● Identical to glDrawArraysInstancedBaseInstance with *baseInstance* set to 0.

Figure 4: Table of the non-indexed, instanced drawing commands in OpenGL[4,6,7]

Non-indexed drawing commands are a subset of the OpenGL drawing commands where the vertex data is stored in buffers in order, and simply fed to the vertex shader[4]. OpenGL provides two non-indexed instanced drawing functions. First, *glDrawArraysInstanced*, which is identical to *glDrawArrays* except you can specify how many sets of the geometry you want to render with the *instancecount* parameter [6]. The use case for this would be very similar to the example previously shown, with the only difference being that your data is to be set up in a non-indexed manner. Furthermore *glDrawArraysInstancedBaseInstance* has another parameter called *baseinstance* which specifies how far into each instanced vertex buffer you want OpenGL to start reading at [3, 7]. For example if *baseinstance* was set to one, the vertex shader would start reading the vertex data from the second location in any instanced buffer [7]. This could be used in a scenario where you wanted to render a subset of an already defined mesh. You could also store multiple meshes in one vertex buffer and then render whatever one is needed, and use the appropriate *baseinstance* for any instance data associated with it [3]. This would mean a larger buffer on the GPU but it would need to be swapped less often, possibly leading to performance gains.

Overview Of Indexed Instanced Draw Functions

Indexed Draw Function	Notes
glDrawElementsInstancedBaseVertexBaseInstance(GLenum mode GLsizei count, GLenum type, void *indices, GLsizei instancecount, GLint basevertex, GLuint baseinstance);	● Draws instanceCount sets of specified geometry. ● Adds *basevertex* to every indice that is read from an index buffer. ● Retrieves instanced vertex attributes from *baseinstance* location in array. ● does not affect the value of *gl_InstanceID*
glDrawElementsInstancedBaseInstance(GLenum mode, GLsizei count, GLenum type, const void * indices, GLsizei instancecount, GLuint baseinstance);	● Identical to glDrawElementsInstancedBaseVertexBaseInstance with *basevertex* set to 0
glDrawElementsInstancedBaseVertex(GLenum mode, GLsizei count, GLenum type, void *indices, GLsizei instancecount, GLint basevertex);	● Identical to glDrawElementsInstancedBaseVertexBaseInstance with *baseinstance* set to 0
glDrawElementsInstanced(GLenum mode, GLsizei count, GLenum type, const void * indices, GLsizei instancecount);	● Identical to glDrawElementsInstancedBaseVertexBaseInstance with *baseinstance* and *basevertex* set to 0

Figure 5: Table of the indexed, instanced drawing commands in OpenGL[4,8,9,10,11]

Indexed drawing commands are a subset of the OpenGL drawing commands where vertex attributes are read in an order that is specified by an index array [4]. OpenGL provides four indexed instanced drawing functions. First, *glDrawElementsInstanced* is identical to

glDrawElements except you can specify how many instances of the geometry you want to render, as shown in the examples earlier [8]. *glDrawElementsInstancedBaseVertex* is slightly more complex in that you can provide an extra parameter *basevertex* which adds a constant onto every index that is read from the index buffer [10]. That is, the i'th element in a draw call will be taken from location indexArray[i] + *basevertex* in the appropriate arrays [10]. Once again, if the vertex data is set up in a clever manner, this function can be used to draw subsets of a mesh, or a particular mesh from a VBO containing multiple. *glDrawElementsInstancedBaseInstance* is the indexed equivalent to *glDrawArraysInstancedBaseInstance*. That is, instanced attributes start being retrieved at location *baseinstance* in the vertex buffers [9]. Use cases for this function would be the same as *glDrawArraysInstancedBaseInstance* but used when vertex data is set up to be drawn from indices. Finally *glDrawElementsInstancedBaseVertexBaseInstance* provides parameters to be able to combine functionality of both *glDrawElementsInstancedBaseInstance* and *glDrawElementsInstancedBaseVertex* [11]. This is to be used when you want to have the functionality of both mentioned functions. That is, adding a constant, *basevertex*, onto all elements read from the index buffer, and reading instanced attributes starting from an offset, *baseinstance*, into the VBO.

Analysis of Aforementioned Rendering Techniques

The specifications for the computer used in the performance test case is the following. 8 Gigabytes of ram. Intel i5 7600k CPU. GTX 1060 graphics card. For the following tests face culling and depth test were disabled to help control the variables that would affect performance. As seen in the table below there is a clear advantage to using instance rending when possible. The performance gains become relatively greater with the more instances you are trying to render. For example, when rendering 10000 cubes there is about a 4x frames per second (FPS) gain when using instancing, but around a 40x gain when rendering a million cubes compared to the naive approach. Without instancing there started to be noticeable performance hits (FPS lower than 144hz) at around 50 thousand cubes. With instancing a million cubes could be rendered without a noticeable difference. It is worth noting that the second approach mentioned, instancing with a uniform array of matrices, outperformed the naive approach when rendering a small number of cubes. After which this approach was not viable due to lack of uniform memory. This shows the massive performance gains that can be achieved when using instanced rendering.

Number of Cubes	1: Naive Approach (FPS)	2: Uniform Instancing (FPS)	3: Instancing with VBO (FPS)	1 vs 3 Relative FPS gain
225	4100	4600	4600	1.1x
10000	630	N/A	2600	4.1x
50625	130	N/A	1650	12.7x
160000	40	N/A	900	22.5x

1000000	6	N/A	220	36.6x

Figure 6: Approximate FPS measured using various rendering approaches.

Conclusion

GPU instancing is a rendering technique that allows the efficient rendering of objects with the same geometry and graphics pipeline setup, but with various transformations. This is useful in games that have repetitive geometry like grass or trees. This allows all the data needed to render identical geometry to be sent to the GPU at once, leading to performance gains. The main approach discussed was to use instanced vertex attributes which allows specified vertex data to be sent to the shaders that updates per instance as opposed to per vertex. The performance gains to this approach become relatively greater with the higher number of instances that are being rendered. It was shown in the case study that there was nearly a 40x fps gain when rendering a million cubes compared to the naive approach.

Appendix

[A]

```
// bind matrix vbo
glBindBuffer(GL_ARRAY_BUFFER, m_matriceVbo);
glVertexAttribPointer(
    matriceAttrib, // Attribute location
    4, // Number of components
    GL_FLOAT, // Type of each component
    GL_FALSE, // Normalize?
    sizeof(GLfloat) * 4 * 4, // Stride
    0); // Offset
glVertexAttribPointer(
    matriceAttrib + 1, // Attribute location
    4, // Number of components
    GL_FLOAT, // Type of each component
    GL_FALSE, // Normalize?
    sizeof(GLfloat) * 4 * 4, // Stride
    (void*)(sizeof(GL_FLOAT) * 4)); // Offset
glVertexAttribPointer(
    matriceAttrib + 2, // Attribute location
    4, // Number of components
    GL_FLOAT, // Type of each component
    GL_FALSE, // Normalize?
    sizeof(GLfloat) * 4 * 4, // Stride
    (void*)(sizeof(GL_FLOAT) * 8)); // Offset
glVertexAttribPointer(
    matriceAttrib + 3, // Attribute location
    4, // Number of components
    GL_FLOAT, // Type of each component
    GL_FALSE, // Normalize?
    sizeof(GLfloat) * 4 * 4, // Stride
    (void*)(sizeof(GL_FLOAT) * 12)); // Offse

glEnableVertexAttribArray(matriceAttrib);
glEnableVertexAttribArray(matriceAttrib + 1);
glEnableVertexAttribArray(matriceAttrib + 2);
glEnableVertexAttribArray(matriceAttrib + 3);
glVertexAttribDivisor(matriceAttrib, 1);
glVertexAttribDivisor(matriceAttrib + 1, 1);
glVertexAttribDivisor(matriceAttrib + 2, 1);
glVertexAttribDivisor(matriceAttrib + 3, 1);
```

Figure 7: Full code example of setting up a 4x4 matrix as an instanced vertex attribute [12].

Work Cited

[1] *Instancing*. LearnOpenGL. (n.d.). https://learnopengl.com/Advanced-OpenGL/Instancing

[2]*glVertexAttribDivisor*. GlVertexAttribDivisor - OpenGL 4 reference pages. (n.d.). https://registry.khronos.org/OpenGL-Refpages/gl4/html/glVertexAttribDivisor.xhtml

[3]Zzzhhh. (2017, December 21). *Confusion in understanding the behavior of Gldrawarraysinstancedbaseinstance*. Khronos Forums. https://community.khronos.org/t/confusion-in-understanding-the-behavior-of-gldrawarraysinstancedbaseinstance/76429

[4] Pearson. (2016). Vertex Processing and Drawing Commands. In *OpenGL SuperBible* (pp. 249–269). Essay.

[5]*glVertexAttribPointer*. Glvertexattribpointer - opengl 4 reference pages. (n.d.). https://registry.khronos.org/OpenGL-Refpages/gl4/html/glVertexAttribPointer.xhtml

[6]*glDrawArraysInstanced* . Gldrawarraysinstanced - OpenGL 4 reference pages. (n.d.). https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawArraysInstanced.xhtml

[7]*glDrawArraysInstancedBaseInstance*. Gldrawarraysinstancedbaseinstance - OpenGL 4 reference pages. (n.d.). https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawArraysInstancedBaseInstance.xhtml

[8]glDrawElementsInstanced. Gldrawelementsinstanced - OpenGL 4 reference pages. (n.d.). https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawElementsInstanced.xhtm

[9]*glDrawElementsInstancedBaseInstance*. Gldrawelementsinstancedbaseinstance - OpenGL 4 reference pages. (n.d.). https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawElementsInstancedBaseInstance.xhtml

[10] *glDrawElementsInstancedBaseVertex*. Gldrawelementsinstancedbasevertex - OpenGL 4 reference pages. (n.d.). https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawElementsInstancedBaseVertex.xhtml

[11]*glDrawElementsInstancedBaseVertexBaseInstance*. Gldrawelementsinstancedbasevertexbaseinstance - OpenGL 4 reference pages. (n.d.). https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawElementsInstancedBaseVertexBaseInstance.xhtml

[12]BramBram 7, & Robert RouhaniRobert Rouhani 14.6k66 gold badges4545 silver badges5959 bronze badges. (2013, July 1). *Using a matrix as vertex attribute in opengl3 core profile*. Stack Overflow. https://stackoverflow.com/questions/17355051/using-a-matrix-as-vertex-attribute-in-opengl3-core-profile